

SPRING BOOT

```
package com.example.springweb;

public class Student {

    private int id;
    private String name;
    private int age ;

    public Student(int id, String name, int age) {
        this.id = id;
        this.name = name;
        this.age = age;
    }

    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public int getAge() {
        return age;
    }
    public void setAge(int age) {
        this.age = age;
    }
}
```

StudentController.java

```
package com.example.springweb;

import java.util.List;
```

```
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

@RestController // controller + requestbody
public class StudentController {

    // http methods

    // get , post, put, delete, patch

    StudentService service;
    public StudentController(StudentService service){
        this.service = service;
    }

    @GetMapping("/home")
    public List<Student> sayHello(){
        return service.getAllStudent();
    }

    // @GetMapping("/")
    // public String Login(){
    //     return "Login";
    // }

    @PostMapping("/add")
    public Student addStudent(@RequestBody Student student){
        System.out.println(student.getId());
        System.out.println(student.getAge());
        System.out.println(student.getName());
        return service.addStudent(student);
    }

    @GetMapping("find/{id}")
    public Student getById(@PathVariable int id){
        return service.getStudentById(id);
    }

    @PutMapping("/{id}")
    public Student update(@PathVariable int id , @RequestBody Student
student){
```

```

        return service.updateStudent(id, student);
    }

    @DeleteMapping("/{id}")
    public String delete(@PathVariable int id){
        return service.deleteStudent(id) ? "Deleted Successfully" : "Student
not found";
    }
}

```

StudentService.java

```

package com.example.springweb;

import java.util.List;

import org.springframework.stereotype.Service;

@Service
public class StudentService {

    private StudentRepository repo;

    // DI
    public StudentService(StudentRepository repo){
        this.repo = repo;
    }

    public List<Student> getAllStudent(){
        return repo.findAll();
    }

    public Student addStudent(Student student){

        return repo.save(student);
    }

    public Student getStudentById(int id){
        return repo.findById(id);
    }

    public Student updateStudent(int id , Student student){

```

```

        return repo.update(id, student);
    }

    public boolean deleteStudent(int id){
        return repo.delete(id);
    }
}

```

studentRepository.java

```

package com.example.springweb;

import java.util.ArrayList;
import java.util.List;

import org.springframework.stereotype.Repository;

@Repository // IOC
public class StudentRepository {

    private List<Student> ls = new ArrayList<>();

    public StudentRepository(){
        ls.add(new Student(1, "FIIT", 23));
        ls.add(new Student(2, "Java", 40));
    }

    public List<Student> findAll(){
        return ls;
    }

    public Student save(Student student){
        ls.add(student);

        return student;
    }

    public Student findById(int id){
        return ls.stream().filter(s -> s.getId() == id)
            .findFirst().orElse(null);
    }

    public Student update(int id, Student updated){

        Student student = findById(id);
    }
}

```

```

        if(student != null){

            student.setName(updated.getName());
            student.setAge(updated.getAge());

        }

        return student;
    }

    public boolean delete(int id){
        return ls.removeIf(s-> s.getId() == id);
    }
}

```

Theory

1. Model Layer

- **POJO (Plain Old Java Object)** → Student class with fields, constructors, getters, and setters.
- Represents the data structure.
- Can later be annotated with `@Entity` if connected to a database.

2. Repository Layer

- **@Repository** → Marks the class as a bean responsible for data access.
- Here you used an in-memory `List<Student>` to simulate persistence.
- Provides methods like `findAll()`, `findById()`, `save()`, `update()`, `delete()`.

3. Service Layer

- **@Service** → Encapsulates business logic.
- Delegates persistence operations to the repository.
- Keeps controllers clean and focused on request handling.

4. Controller Layer

- **@RestController** → Combines `@Controller` and `@ResponseBody`.

- Handles HTTP requests (GET, POST, PUT, DELETE).
- Uses annotations like `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`.
- **Dependency Injection** → Constructor injection of `StudentService`.

Coding Round Questions

- Controller → What is the role of `@RestController`?
- Service → Why do we use `@Service`?
- Repository → What is the role of `@Repository`?
- Dependency Injection → Why is constructor injection preferred?
- CRUD → How do you implement CRUD operations in Spring Boot?
- POJO → What is a POJO and why is it important?

Machine Coding Round Challenge

Problem: Student CRUD API with Validation

Requirements:

1. Extend your current setup to support full CRUD.
2. Add validation:
 - name must not be empty.
 - age must be greater than 18.
3. Return appropriate HTTP status codes (200 OK, 404 Not Found, 400 Bad Request).
4. Use `ResponseEntity` for responses.